

Vertex AI Pipelines PSC 介面 SWP

來源：<https://codelabs.developers.google.com/pipelines-psc-interface-swp?hl=zh-tw>

步驟數：18

在本教學課程中，您將瞭解如何設定及驗證 Private Service Connect 介面，並透過與 Secure Web Proxy(SWP) 整合的 Vertex AI Pipelines，連線至非 RFC 1918 端點。

目錄

1. 簡介
2. 事前準備
3. 消費者設定
4. 啟用 IAP
5. 建立消費者 VM 執行個體
6. Secure Web Proxy
7. Private Service Connect 網路連結
8. 私人 DNS 區域
9. 建立 Jupyter Notebook
10. 建立 Vertex AI Workbench 執行個體
11. 更新 Vertex AI 服務代理
12. 在「class-e-vm」上安裝 Apache2 並啟用 Tcpdump :
13. 部署 Vertex AI Pipelines 工作
14. PSC 介面驗證
15. Cloud Logging 驗證
16. TCPDump 驗證
17. 清理
18. 恭喜

程式碼研究室簡介

subject上次更新時間：3月 27, 2026

account_circle作者：Harika Rama Tulasi Karatapu, Deepak Michael

1. 簡介

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶虛擬私有雲網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

如果網路連結接受來自 Private Service Connect 介面的連線，Google Cloud 會從網路連結指定的消費者子網路，為介面分配 IP 位址。消費者和生產者網路已連線，可使用內部 IP 位址通訊。

網路連結與 Private Service Connect 介面之間的連線，類似於 Private Service Connect 端點與服務連結之間的連線，但有兩項主要差異：

- 網路連結可讓供應商網路啟動與消費者網路的連線 (代管服務輸出)，而端點則可讓消費者網路啟動與供應商網路的連線 (代管服務輸入)。
- Private Service Connect 介面連線是可遞移的。也就是說，生產端網路可以與連線至消費端網路的其他網路通訊。

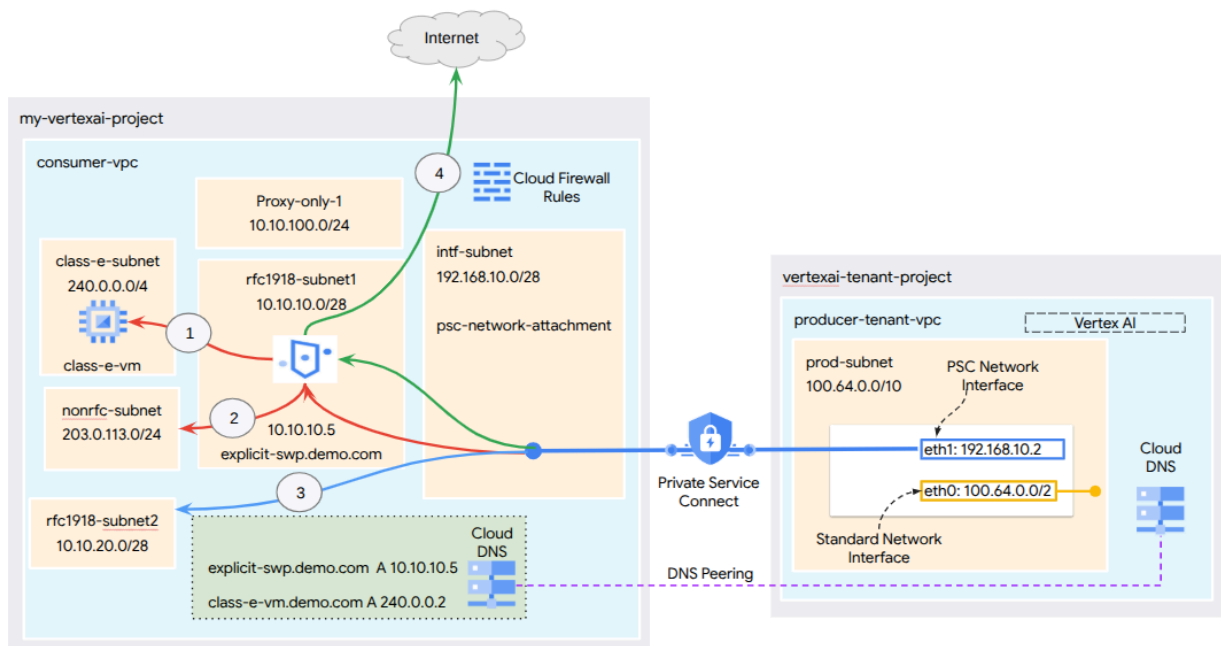


圖 1

Vertex AI PSC 介面可連線性考量

- Vertex AI PSC 介面可將流量路由至 RFC1918 位址區塊內的虛擬私有雲或地端部署目的地。
- 如果 PSC 介面指定非 RFC-1918 位址區塊，則必須在消費者端的虛擬私有雲中部署 RFC-1918 位址的明確 Proxy。在 Vertex AI 部署作業中，必須定義 Proxy，以及目標端點的 FQDN。請參閱圖 1，瞭解在消費者 VPC 中設定的明確 Proxy 模式安全網路 Proxy (SWP)，如何協助將路由傳送至下列非 RFC-1918 CIDR：

- 240.0.0.0/4
- 203.0.113.0/24
- 10.10.20.0/28 不需要 Proxy，屬於 RFC-1918 範圍。
- 網際網路輸出

Google 管理的租戶網路的網際網路連線：

沒有 VPC-SC 的 Vertex AI PSC 介面

- 如果您只使用 PSC 介面設定部署作業，系統會保留預設的網際網路存取權。這類傳出流量會直接從 Google 代管的租戶網路輸出。

搭配使用 Vertex AI PSC 介面與 VPC-SC

- 如果專案屬於 VPC Service Controls perimeter，perimeter 會封鎖 Google 代管租戶的預設網路連線能力，防止資料竊取。
- 如要允許部署作業在此情境中存取公開網際網路，您必須明確設定安全輸出路徑，將流量透過連線至 Vertex AI 的 VPC 傳送。在虛擬私有雲網路中部署 RFC 1918 位址的 Proxy 伺服器，並與 Cloud NAT 閘道配對，是其中一種做法。請注意，您也可以使用 Secure Web Proxy 將流量轉送至網際網路。建立 Secure Web Proxy 時，系統會自動建立 Cloud NAT 閘道。

注意：使用 VPC-SC 時，Vertex AI Pipelines 部署作業需要明確的網際網路輸出 Proxy。如果未啟用 VPC-SC，系統會直接透過 Google 管理的租戶 VPC 提供網際網路輸出。

如需更多資訊，請參閱下列資源：

為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud

建構項目

在本教學課程中，您將建構完整的 Vertex AI Pipelines 部署作業，並使用 Private Service Connect (PSC) 介面，允許生產端連線至消費者的運算資源，如圖 1 所示，目標是 `class-e-subnet` 中非 RFC 1918 的端點。

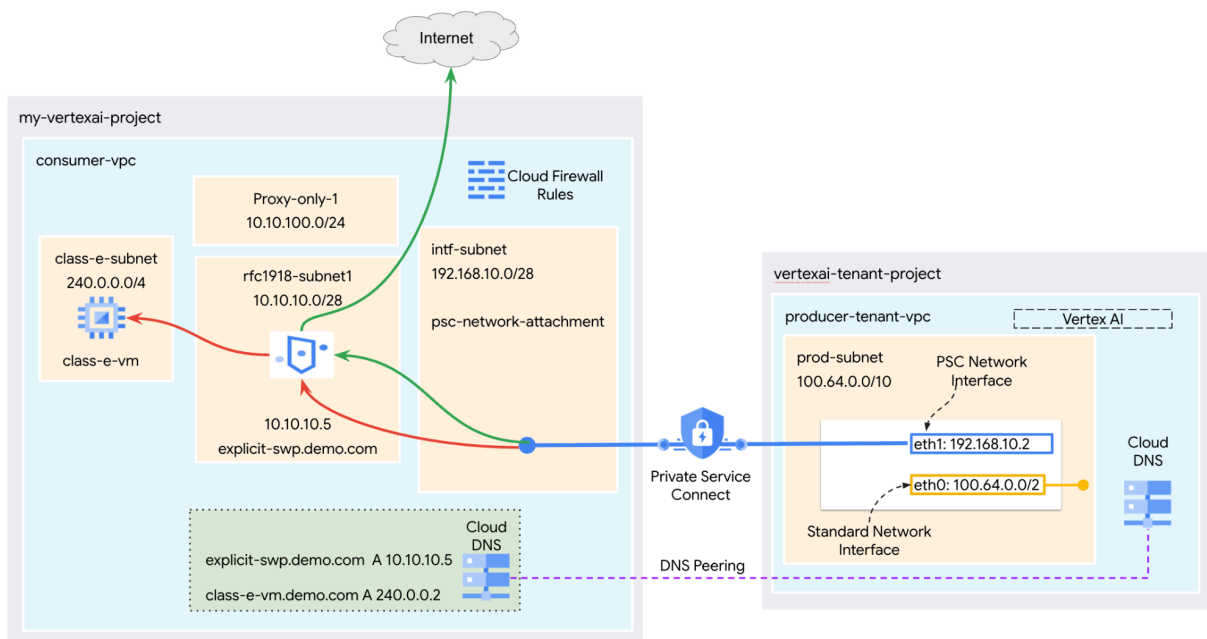


圖 2

您將在用戶 VPC 中建立單一 `psc-network-attachment`，利用 DNS 對接功能解析代管 Vertex AI 訓練 的租戶專案中的用戶 VM，進而實現下列用途：

以明確的 Proxy 模式部署 Vertex AI Pipelines 並設定 Secure Web Proxy，允許對 E 類子網路中的 VM 執行 `wget`。

注意：本教學課程提供的設定和驗證步驟，是根據使用獨立 VPC 的圖示拓撲，請視貴機構的需求修改程序。

課程內容

- 如何建立網路連結
- 生產者如何使用網路連結建立 PSC 介面
- 如何建立 DNS 對等互連，從 Google 代管虛擬私有雲網路解析在消費者虛擬私有雲網路中設定的私人網域
- 如何將 Vertex AI PSC 介面中的流量轉送至 Secure Web Proxy
- 如何從 Vertex AI Pipelines 建立與非 RFC-1918 IP 位址空間的通訊

軟硬體需求

Google Cloud 專案

IAM 權限

- [Compute 網路管理員](#) (roles/compute.networkAdmin)
 - [Compute 執行個體管理員](#) (roles/compute.instanceAdmin)
 - [Compute 安全管理員](#) (roles/compute.securityAdmin)
 - [安全性政策管理員](#) (roles/compute.orgSecurityPolicyAdmin)
 - [DNS 管理員](#) (roles/dns.admin)
 - [受 IAP 保護的通道使用者](#) (roles/iap.tunnelResourceAccessor)
 - [記錄管理員](#) (roles/logging.admin)
 - [Notebooks 管理員](#) (roles/notebooks.admin)
 - [專案 IAM 管理員](#) (roles/resourcemanager.projectIamAdmin)
 - [服務帳戶管理員](#) (roles/iam.serviceAccountAdmin)
 - [服務帳戶使用者](#) (roles/iam.serviceAccountUser)
 - [服務使用情形管理員](#) (roles/serviceusage.serviceUsageAdmin)
-

步驟 2 · 約 2 分鐘

2. 事前準備

更新專案以支援教學課程

本教學課程會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-ID]
projectid=YOUR-PROJECT-ID
echo $projectid
```

啟用 API

在 Cloud Shell 中執行下列操作：

```
gcloud services enable "compute.googleapis.com"
gcloud services enable "aiplatform.googleapis.com"
gcloud services enable "dns.googleapis.com"
gcloud services enable "notebooks.googleapis.com"
gcloud services enable "storage.googleapis.com"
gcloud services enable "cloudresourcemanager.googleapis.com"
gcloud services enable "artifactregistry.googleapis.com"
gcloud services enable "cloudbuild.googleapis.com"
gcloud services enable "networkservices.googleapis.com"
gcloud services enable "networksecurity.googleapis.com"
gcloud services enable "certificatemanager.googleapis.com"
```

步驟 3 · 約 10 分鐘

3. 消費者設定

建立 Consumer VPC

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create consumer-vpc --project=$projectid --subnet-mode=custom
```

建立消費者子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create class-e-subnet --project=$projectid --range=240.0.0.0/4 --network=consumer-vpc --region=us-central1
```

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create rfc1918-subnet1 --project=$projectid --range=10.10.10.0/28 --network=consumer-vpc --region=us-central1 --enable-private-ip-google-access
```

建立僅限 Proxy 的子網路

```
gcloud compute networks subnets create proxy-only-uscentral1 \
  --purpose=REGIONAL_MANAGED_PROXY \
  --role=ACTIVE \
  --region=us-central1 \
  --network=consumer-vpc \
  --range=10.10.100.0/26
```

注意：IP 位址範圍 (`--range`) 必須至少為 **/26** (64 個 IP 位址)。這個子網路中的位址僅供 Google 管理的 Proxy 使用，無法用於 VM 或其他資源。

建立 Private Service Connect 網路連結子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create intf-subnet \
  --project=$projectid \
  --range=192.168.10.0/28 \
  --network=consumer-vpc \
  --region=us-central1 \
  --enable-private-ip-google-access
```

Cloud Router 和 NAT 設定

Google Cloud Secure Web Proxy 會在部署的區域中，自動佈建及管理 Cloud NAT 閘道和相關聯的 Cloud Router。

步驟 4 · 約 5 分鐘

4. 啟用 IAP

如要允許 IAP (Identity-Aware Proxy) 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

在 Cloud Shell 中，建立 IAP 防火牆規則。

```
gcloud compute firewall-rules create ssh-iap-consumer \  
  --network consumer-vpc \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

步驟 5 · 約 0 分鐘

5. 建立消費者 VM 執行個體

在 Cloud Shell 中建立消費者 VM 執行個體 `class-e-vm`。

```
gcloud compute instances create class-e-vm \  
  --project=$projectid \  
  --machine-type=e2-micro \  
  --image-family debian-11 \  
  --no-address \  
  --shielded-secure-boot \  
  --image-project debian-cloud \  
  --zone us-central1-a \  
  --subnet=class-e-subnet \  
  --private-network-ip=240.0.0.2
```

步驟 6 · 約 0 分鐘

6. Secure Web Proxy

安全網路 Proxy 的明確模式 (或明確 Proxy 轉送模式) 是一種部署方法，必須明確設定用戶端工作負載，才能將 SWP 的內部 IP 位址或完整網域名稱和連接埠做為轉送 Proxy。

在下列步驟中，請務必將 `YOUR-PROJECT-ID` 修改為您的專案 ID

建立網路 Proxy：

在 Cloud Shell 中，使用文字編輯器建立 `policy.yaml` 檔案：

```
cat > policy.yaml << EOF
description: basic Secure Web Proxy policy
name: projects/$projectid/locations/us-central1/gatewaySecurityPolicies/policy1
EOF
```

在 Cloud Shell 中產生 Secure Web Proxy 政策：

```
gcloud network-security gateway-security-policies import policy1 \
  --source=policy.yaml \
  --location=us-central1
```

在下一節中，建立規則以允許根據主機 `sessionMatcher` 存取 `class-e-vm`。

在 Cloud Shell 中，使用文字編輯器建立 `rule1.yaml` 檔案：

```
cat > rule1.yaml << EOF
name: projects/$projectid/locations/us-central1/gatewaySecurityPolicies/policy1/rules/allow-nonrfc-classe
description: Allow nonrfc class-e
enabled: true
priority: 1
basicProfile: ALLOW
sessionMatcher: host() == 'class-e-vm.demo.com'
EOF
```

在下一節中，請建立規則，允許 Jupyter Notebook 存取 `"class-e"` VM，以便安裝 `apache2`。

在 Cloud Shell 中，使用文字編輯器建立 `rule2.yaml` 檔案：

```
cat > rule2.yaml << EOF
name: projects/$projectid/locations/us-central1/gatewaySecurityPolicies/policy1/rules/allow-apache2
description: Allow Apache2 install on class-e VM
enabled: true
priority: 2
basicProfile: ALLOW
sessionMatcher: inIpRange(source.ip, '240.0.0.2')
EOF
```

在 Cloud Shell 中產生安全性政策規則 1：

```
gcloud network-security gateway-security-policies rules import allow-nonrfc-classe \  
  --source=rule1.yaml \  
  --location=us-central1 \  
  --gateway-security-policy=policy1
```

在 Cloud Shell 中，產生安全性政策規則 2：

```
gcloud network-security gateway-security-policies rules import allow-apache2 \  
  --source=rule2.yaml \  
  --location=us-central1 \  
  --gateway-security-policy=policy1
```

如要支援 Vertex AI 訓練，請使用下列設定檔設定 Secure Web Proxy 閘道：

- 接聽埠：使用在 Vertex AI 應用程式的程式碼明確 Proxy 設定中設定的相同通訊埠 (例如 8080)。
- 位址：從 RFC 1918 範圍指派私人 IP 位址。
- 轉送模式：將此模式設為 EXPLICIT_ROUTING_MODE

在 Cloud Shell 中，建立 gateway.yaml 檔案來定義 Secure Web Proxy 閘道：

```
cat > gateway.yaml << EOF  
name: projects/$projectid/locations/us-central1/gateways/swp1  
type: SECURE_WEB_GATEWAY  
addresses: ["10.10.10.5"]  
ports: [8080]  
gatewaySecurityPolicy: projects/$projectid/locations/us-  
central1/gatewaySecurityPolicies/policy1  
network: projects/$projectid/global/networks/consumer-vpc  
subnetwork: projects/$projectid/regions/us-central1/subnetworks/rfc1918-subnet1  
routingMode: EXPLICIT_ROUTING_MODE  
EOF
```

在 Cloud Shell 中，產生 Secure Web Proxy 執行個體：

```
gcloud network-services gateways import swp1 \  
  --source=gateway.yaml \  
  --location=us-central1
```

部署 Secure Web Proxy 可能需要幾分鐘的時間。



Web proxy details

[Edit](#)[Delete](#)

swp1 

Description	—
Routing mode	Explicit
Region	us-central1
Network	consumer-vpc
Subnetwork	rfc1918-subnet1
Web proxy IP address	10.10.10.5
Ports	8080
Certificate	—
Associated policy	policy1
Created	Oct 6, 2025, 11:50:50 PM
Last updated	Oct 6, 2025, 11:50:58 PM

7. Private Service Connect 網路連結

網路連結是區域資源，代表 Private Service Connect 介面的用戶端。您會將單一子網路與網路連結建立關聯，而生產端會從該子網路將 IP 指派給 Private Service Connect 介面。子網路必須與網路連結位於同一地區。網路連結必須與生產者服務位於相同區域。

建立網路連結

在 Cloud Shell 中建立網路連結。

```
gcloud compute network-attachments create psc-network-attachment \
  --region=us-central1 \
  --connection-preference=ACCEPT_MANUAL \
  --subnets=intf-subnet
```

注意：設定 Vertex AI 時，系統會自動新增 Google 管理的租戶專案，因此您不必在附件中明確提及接受的專案 ID，就像設定「自動接受」一樣。

列出網路連結

在 Cloud Shell 中列出網路連結。

```
gcloud compute network-attachments list
```

說明網路連結

在 Cloud Shell 中，說明網路附件。

```
gcloud compute network-attachments describe psc-network-attachment --region=us-central1
```

請記下 PSC 網路連結名稱 `psc-network-attachment`，供應商建立 Private Service Connect 介面時會用到這個名稱。

如要在 Cloud 控制台中查看 PSC 網路附件網址，請前往下列位置：

「網路服務」→「Private Service Connect」→「網路連結」→「psc-network-attachment」

← Network attachment details

Edit

Delete

psc-network-attachment

Network attachment	projects/[REDACTED]/regions/us-central1/networkAttachments/psc-network-attachment
Region	us-central1
Network	consumer-vpc
Subnetworks	intf-subnet
Connection preference	Accept manually
Tags	—

步驟 8 · 約 3 分鐘

8. 私人 DNS 區域

您將為 `demo.com` 建立 Cloud DNS 區域，並填入指向 VM IP 位址的 A 記錄。稍後，DNS 對等互連會部署在 Vertex AI Pipelines 工作中，以便存取消費者的 DNS 記錄。

在 Cloud Shell 中執行下列操作：

```
gcloud dns --project=$projectid managed-zones create private-dns-codelab --description="" --
dns-name="demo.com." --visibility="private" --
networks="https://compute.googleapis.com/compute/v1/projects/$projectid/global/networks/consumer-vpc"
```

在 Cloud Shell 中，為 VM 建立記錄集 `class-e-vm`，並根據環境的輸出內容更新 IP 位址。

```
gcloud dns --project=$projectid record-sets create class-e-vm.demo.com. --zone="private-dns-codelab" --type="A" --ttl="300" --rrdatas="240.0.0.2"
```

在 Cloud Shell 中，為 Secure Web Proxy 建立記錄集，並根據環境的輸出內容更新 IP 位址。

```
gcloud dns --project=$projectid record-sets create explicit-swp.demo.com. --zone="private-dns-codelab" --type="A" --ttl="300" --rrdatas="10.10.10.5"
```

建立 Cloud Firewall 防火牆規則，允許從 PSC 介面存取

在下一節中，請建立防火牆規則，允許來自 PSC 網路附件的流量存取消費者虛擬私有雲中的 RFC 1918 計算資源。

在 Cloud Shell 中，建立允許從僅限 Proxy 的子網路存取 E 類子網路的輸入防火牆規則。因為 SWP 會以僅限 Proxy 的子網路做為來源位址，啟動與該子網路的連線。

```
gcloud compute firewall-rules create allow-access-to-class-e \
  --network=consumer-vpc \
  --action=ALLOW \
  --rules=ALL \
  --direction=INGRESS \
  --priority=1000 \
  --source-ranges="10.10.100.0/28" \
  --destination-ranges="240.0.0.0/4" \
  --enable-logging
```

9. 建立 Jupyter Notebook

下一節將引導您建立 Jupyter Notebook。這個筆記本會用於部署 Vertex AI Pipelines 工作，將 `wget` 從 Vertex AI Pipelines 傳送至測試執行個體。Vertex AI Pipelines 與含有執行個體的消費者網路之間的資料路徑，會使用 Private Service Connect 介面。

建立使用者管理的服務帳戶

在下一節中，您將建立與本教學課程所用 Vertex AI Workbench 執行個體相關聯的服務帳戶。

在本教學課程中，服務帳戶會套用下列角色：

- [儲存空間管理員](#)
- [Vertex AI 使用者](#)
- [Artifact Registry 管理員](#)
- [Cloud Build 編輯者](#)
- [IAM 服務帳戶使用者](#)

登入 Cloud Shell 並執行下列操作：

建立服務帳戶。

```
gcloud iam service-accounts create notebook-sa \  
  --display-name="notebook-sa"
```

將服務帳戶更新為 Storage 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/storage.admin"
```

將服務帳戶更新為 AI Platform 使用者角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/aiplatform.user"
```

將 Artifact Registry 管理員角色授予服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/artifactregistry.admin"
```

將 Cloud Build 編輯者角色授予服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/cloudbuild.builds.editor"
```

允許筆記本服務帳戶使用 Compute Engine 預設服務帳戶。


```
gcloud iam service-accounts add-iam-policy-binding \  
    $(gcloud projects describe $(gcloud config get-value project) --  
format='value(projectNumber)')-compute@developer.gserviceaccount.com \  
    --member="serviceAccount:notebook-sa@$projectid.iam.gserviceaccount.com" \  
    --role="roles/iam.serviceAccountUser"
```

步驟 10 · 約 5 分鐘

10. 建立 Vertex AI Workbench 執行個體

在下一節中，建立納入先前建立的服務帳戶 `notebook-sa` 的 Vertex AI Workbench 執行個體。

在 Cloud Shell 中建立私有用戶端執行個體。

```
gcloud workbench instances create workbench-tutorial --vm-image-project=cloud-notebooks-managed --vm-image-family=workbench-instances --machine-type=n1-standard-4 --location=us-central1-a --subnet-region=us-central1 --subnet=rfc1918-subnet1 --disable-public-ip --shielded-secure-boot=true --shielded-integrity-monitoring=true --shielded-vtpm=true --service-account-email=notebook-sa@$projectid.iam.gserviceaccount.com
```

11. 更新 Vertex AI 服務代理

Vertex AI 會代表您執行作業，例如從用於建立 PSC 介面的 PSC 網路連結子網路取得 IP 位址。為此，Vertex AI 會使用[服務代理](#) (如下所列)，該代理需要[網路管理員](#)權限。

`service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com`

注意：更新服務代理權限前，請先前往 Cloud 控制台的 Vertex AI，確認已啟用 Vertex AI API。

在 **Cloud Shell** 中：

取得專案編號。

```
gcloud projects describe $projectid | grep projectNumber
```

設定專案編號。

```
projectnumber=YOUR-PROJECT-NUMBER
```

為 AI Platform 建立服務帳戶。如果專案中已有服務帳戶，請略過這個步驟。

```
gcloud beta services identity create --service=aiplatform.googleapis.com --  
project=$projectnumber
```

將 `compute.networkAdmin` 角色指派給服務代理帳戶。

```
gcloud projects add-iam-policy-binding $projectid --  
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --  
role="roles/compute.networkAdmin"
```

使用 `dns.peer` 角色更新服務代理程式帳戶

```
gcloud projects add-iam-policy-binding $projectid --  
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --  
role="roles/dns.peer"
```

更新預設服務帳戶

啟用 Compute Engine API，並[授予預設服務帳戶](#) Vertex AI 存取權。請注意，存取權變更可能需要一段時間才會生效。

使用 Cloud Shell 更新預設服務帳戶，步驟如下：

將預設服務帳戶更新為 `aiplatform.user` 角色

```
gcloud projects add-iam-policy-binding $projectid \  
--member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
--role="roles/aiplatform.user"
```

將預設服務帳戶更新為 `storage.admin` 角色

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
  --role="roles/storage.admin"
```

將預設服務帳戶更新為 `artifactregistry.admin` 角色

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
  --role="roles/artifactregistry.admin"
```

步驟 12 · 約 5 分鐘

12. 在「class-e-vm」上安裝 Apache2 並啟用 Tcpdump：

透過 Secure Web Proxy 安裝 apache2：`class-e-vm`

開啟新的 Cloud Shell 分頁，更新專案變數，然後透過 SSH 連線至 class-e-vm

```
gcloud compute ssh --zone us-central1-a "class-e-vm" --tunnel-through-iap --project $projectid
```

```
sudo apt-get -o Acquire::http::Proxy="http://10.10.10.5:8080" update
sudo apt-get -o Acquire::http::Proxy="http://10.10.10.5:8080" install apache2 -y
sudo service apache2 restart
echo 'class-e Server !!' | sudo tee /var/www/html/index.html
```

在僅限 Proxy 的子網路上執行 `tcpdump` 篩選作業，Secure Web Proxy 會使用這個子網路將流量轉送至目標。

從 class-e-vm OS 執行 tcpdump，篩選 proxy-vm 子網路。

```
sudo tcpdump -i any net 10.10.100.0/24 -nn
```

注意：請務必在 workbench-tutorial 教學課程執行個體子網路上啟用 Private Google Access，才能開啟 JupyterLab 工作階段。

13. 部署 Vertex AI Pipelines 工作

在下一節中，您將建立筆記本，從 Vertex AI Pipelines 成功執行 `wget` 至明確的 Proxy。這樣一來，您就能連線至非 RFC 1918 VM，例如 `class-e-vm`。Vertex AI Pipelines 的目標是 RFC 1918 IP 位址，因此不需明確的 Proxy 即可存取 `rfc1918-vm`。

在 Vertex AI Workbench 執行個體中執行訓練工作。

1. 在 Google Cloud 控制台中，前往 Vertex AI Workbench 頁面的執行個體分頁。
2. 按一下 Vertex AI Workbench 執行個體名稱 (workbench-tutorial) 旁的「Open JupyterLab」。Vertex AI Workbench 執行個體會 JupyterLab 中開啟。
3. 依序選取「檔案」>「新增」>「筆記本」
4. 選取「Kernel」>「Python 3」

在 JupyterLab 筆記本中建立新儲存格，然後更新並執行下列項目。請務必將 `PROJECT_ID` 更新為環境詳細資料。

```
import json
import requests
import pprint

PROJECT_ID = 'YOUR-PROJECT-ID' #Enter your project ID
PROJECT_NUMBER=!gcloud projects list --filter="project_id:$PROJECT_ID" --
format="value(PROJECT_NUMBER)"
PROJECT_NUMBER=str(PROJECT_NUMBER).strip('[').strip(']').strip('"')
print(PROJECT_NUMBER)
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# us-central1 is used for the codelab
REGION = "us-central1" #@param {type:"string"}
SERVICE_NAME = "aiplatform" #@param {type:"string"}
SERVICE = "{}.googleapis.com".format(SERVICE_NAME)
ENDPOINT="{}-{}.googleapis.com".format(REGION, SERVICE_NAME)
API_VERSION = "v1" # @param {type: "string"}

LOCATION = REGION
```

在 JupyterLab 筆記本中建立新儲存格，然後執行下列設定，並注意以下重點：

- `proxy_server = "http://explicit-swp.demo.com:8080"`
- FQDN 會與部署在用戶虛擬私有雲中的 Proxy VM 建立關聯。我們會在後續步驟中使用 DNS 對等互連來解析 FQDN。

```

%%writefile main.py

import logging
import socket
import sys
import os

def make_api_request(url: str, proxy_vm_ip: str, proxy_vm_port: str):
    """
    Makes a GET request to a nonRFC-1918 API and saves the response.

    Args:
        url: The URL of the API to send the request to.
    """
    import requests

    try:
        # response = requests.get(url)
        proxy_server = f"http://explicit-swp.demo.com:8080" # replace it with your Secure Web
        proxy Ip-address and the port.

        proxies = {
            "http": proxy_server,
            "https": proxy_server,
        }

        response = requests.get(url, proxies=proxies)
        logging.info(response.text)

        response.raise_for_status() # Raise an exception for bad status codes
        logging.info(f"Successfully fetched data from {url}")
    except requests.exceptions.RequestException as e:
        logging.error(f"An error occurred: {e}")
        raise e

if __name__ == '__main__':
    # Configure logging to print clearly to the console
    logging.basicConfig(
        level=logging.INFO,
        format='%(levelname)s: %(message)s',
        stream=sys.stdout
    )
    url_to_test = os.environ['NONRFC_URL']
    proxy_vm_ip = os.environ['PROXY_VM_IP']
    proxy_vm_port = os.environ['PROXY_VM_PORT']

    logging.info(f"url_to_test: {url_to_test}")
    logging.info(f"proxy_vm_ip: {proxy_vm_ip}")
    logging.info(f"proxy_vm_port: {proxy_vm_port}")
    make_api_request(url_to_test, proxy_vm_ip, proxy_vm_port)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
%%writefile Dockerfile
FROM python:3.9-slim

RUN apt-get update && \
    apt-get install -y iputils-ping && \
    apt-get install -y wget

RUN pip install cloudml-hypertune requests kfp

COPY main.py /main.py

ENTRYPOINT ["python3", "/main.py"]
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud artifacts repositories create pipelines-test-repo-psc --repository-format=docker --location=us-central1
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
IMAGE_PROJECT = PROJECT_ID
IMAGE_REPO = 'pipelines-test-repo-psc'
IMAGE_NAME = 'nonrfc-ip-call'
TAG = 'v1'

IMAGE_URI= f'us-central1-docker.pkg.dev/{IMAGE_PROJECT}/{IMAGE_REPO}/{IMAGE_NAME}:{TAG}'
IMAGE_URI
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud auth configure-docker us-docker.pkg.dev --quiet
```

在 JupyterLab 筆記本中，建立新的儲存格並執行下列程式碼。如有錯誤 (gcloud.builds.submit)，請忽略。

```
!gcloud builds submit --tag {IMAGE_URI} --region=us-central1
```

在 JupyterLab 筆記本中建立並執行下列儲存格，請注意以下重點：

- 使用網域名稱 demo.com 的 dnsPeeringConfigs (dnsPeeringConfigs)，設定與消費者虛擬私有雲的 DNS 對等互連。
- 這裡的明確轉送模式網路 Proxy 是 explicit-swp.demo.com。解析作業會在消費者 VPC 內透過 DNS 對接處理。
- 通訊埠 8080 是在 Secure Web Proxy 中設定的監聽通訊埠 (預設)
- `wget` 會透過 DNS 對等互連解析 class-e-vm-demo.com
- 程式碼會為 Vertex 指定「psc-network-attachment」，讓 Vertex 能使用網路連結子網路部署兩個 PSC 介面執行個體。


```

import json
from datetime import datetime

JOB_ID_PREFIX='test_psci-nonRFC' #@param {type:"string"}
JOB_ID = '{}_{}'.format(JOB_ID_PREFIX, datetime.now().strftime("%Y%m%d%H%M%S"))

# PSC-I configs

PRODUCER_PROJECT_ID = PROJECT_ID
DNS_DOMAIN = 'class-e-vm.demo.com' #@param {type:"string"}
NON_RFC_URL = f"http://{DNS_DOMAIN}"

PROXY_VM_IP = "explicit-swp.demo.com" #@param {type:"string"}
PROXY_VM_PORT = "8080" #@param {type:"string"}

CUSTOM_JOB = {
    "display_name": JOB_ID,
    "job_spec": {
        "worker_pool_specs": [
            {
                "machine_spec": {
                    "machine_type": "n1-standard-4",
                },
                "replica_count": 1,
                "container_spec": {
                    "image_uri": IMAGE_URI,
                    "env": [{
                        "name": "NONRFC_URL",
                        "value": NON_RFC_URL
                    },
                    {
                        "name": "PROXY_VM_IP",
                        "value": PROXY_VM_IP
                    },
                    {
                        "name": "PROXY_VM_PORT",
                        "value": PROXY_VM_PORT
                    }
                ]
            },
        ],
        "enable_web_access": True,
        "psc_interface_config": {
            "network_attachment": "psc-network-attachment",
            "dns_peering_configs": [
                {
                    "domain": "demo.com.",
                    "target_project": PROJECT_ID,
                    "target_network": "consumer-vpc"
                },
            ],
        ],
    },
}

```

```

    },
}
}

print(json.dumps(CUSTOM_JOB, indent=2))

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

import requests
bearer_token = !gcloud auth application-default print-access-token
headers = {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer {}'.format(bearer_token[0]),
}

request_uri = f"https://{REGION}-
aiplatform.googleapis.com/{API_VERSION}/projects/{PROJECT_NUMBER}/locations/{REGION}/customJobs/"

print("request_uri: ", request_uri)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

response_autopush = requests.post(request_uri, json=CUSTOM_JOB, headers=headers)
response = response_autopush
print("response:", response)
if response.reason == 'OK':
    job_name = response.json()['name']
    job_id = job_name.split('/')[-1]
    print("Created Job: ", response.json()['name'])
else:
    print(response.text)

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

# Print KFP SDK version (should be >= 1.6)
! python3 -c "import kfp; print('KFP SDK version: {}'.format(kfp.__version__))"

# Print AI Platform version
! python3 -c "from google.cloud import aiplatform; print('AI Platform version:
{}'.format(aiplatform.__version__))"

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

BUCKET_URI = "your-unique-bucket" # Provide a globally unique bucket name

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
!gcloud storage buckets create gs://{BUCKET_URI}
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# pipeline parameters
CACHE_PIPELINE = False # @param {type: "string"}
_DEFAULT_IMAGE = IMAGE_URI
BUCKET_URI = "gs://{BUCKET_URI}" # @param {type: "string"}
PIPELINE_ROOT = f"{BUCKET_URI}/pipeline_root/intro"
PIPELINE_DISPLAY_NAME = "pipeline_nonRFCIP" # @param {type: "string"}
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

from re import S
import kfp
from kfp import dsl
from kfp.dsl import container_component, ContainerSpec
from kfp import compiler
from google.cloud import aiplatform

# ==== Component with env variable ====

@container_component
def dns_peering_test_op(dns_domain: str, proxy_vm_ip:str, proxy_vm_port:str):
    return ContainerSpec(
        image=_DEFAULT_IMAGE,
        command=["bash", "-c"],
        args=[
            """
            apt-get update && apt-get install inetutils-traceroute inetutils-ping netcat-
openbsd curl -y

            echo "Local IP(s): $(hostname -I)"

            echo "Attempting to trace route to %s"
            traceroute -w 1 -m 7 "%s"

            echo "Sending curl requests to http://%s via proxy %s:%s and recording trace..."
            if curl -L -v --trace-ascii /dev/stdout -x http://%s:%s "http://%s"; then
                echo "Curl request succeeded!"
            else
                echo "Curl request failed!"
                exit 1
            fi
            """ % (dns_domain, dns_domain, dns_domain, proxy_vm_ip, proxy_vm_port,
proxy_vm_ip, proxy_vm_port, dns_domain)

        ]
    )

# ==== Pipeline ====
@dsl.pipeline(
    name="dns-peering-test-pipeline",
    description="Test DNS Peering using env variable",
    pipeline_root=PIPELINE_ROOT,
)
def dns_peering_test_pipeline(dns_domain: str, proxy_vm_ip:str, proxy_vm_port:str):
    dns_test_task = dns_peering_test_op(dns_domain=dns_domain, proxy_vm_ip=proxy_vm_ip,
proxy_vm_port=proxy_vm_port)
    dns_test_task.set_caching_options(enable_caching=CACHE_PIPELINE)

# ==== Compile pipeline ====
if __name__ == "__main__":
    aiplatform.init(project=PROJECT_ID, location=LOCATION)

```

```
compiler.Compiler().compile(  
    pipeline_func=dns_peering_test_pipeline,  
    package_path="dns_peering_test_pipeline.yaml",  
)  
print("✅ Pipeline compiled to dns_peering_test_pipeline.yaml")
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# Define the PipelineJob body; see API Reference https://cloud.google.com/vertex-ai/docs/reference/rest/v1/projects.locations.pipelineJobs/create  
  
import requests, json  
import datetime  
  
bearer_token = !gcloud auth application-default print-access-token  
headers = {  
    'Content-Type': 'application/json',  
    'Authorization': 'Bearer {}'.format(bearer_token[0]),  
}  
  
request_uri = f"https://{REGION}-  
aiplatform.googleapis.com/{API_VERSION}/projects/{PROJECT_NUMBER}/locations/{REGION}/pipeline  
Jobs/"  
  
print("request_uri: ", request_uri)
```

注意：首次執行時，最後一個儲存格執行完畢後，Vertex AI Pipelines 訓練作業可能需要最多 15 分鐘才能完成。如要監控狀態，請前往下列位置：

Vertex AI → 「訓練」 → 「自訂工作」

14. PSC 介面驗證

您也可以前往下列位置，查看 Vertex AI Pipelines 使用的網路附件 IP：

「網路服務」→「Private Service Connect」→「網路連結」→「psc-network-attachment」

選取租戶專案 (專案名稱結尾為「-tp」)

psc-network-attachment

Network attachment	projects/vertex-cosmo/regions/us-central1/networkAttachments/psc-network-attachment 
Region	us-central1
Network	consumer-vpc
Subnetworks	intf-subnet
Connection preference	Accept manually

Connected projects

 Filter	Filter projects			
☐	Name ↑	Status	Connections	Actions
☐	fc04d8d4168daeb73-tp	✔ Accepted	2	⋮

醒目顯示的欄位表示 Vertex AI Pipelines 從 PSC 網路連結使用的 IP 位址。

fc04d8d4168daeb73-tp

Status	✔ Accepted
Connections	2

Endpoints

 Filter	Filter endpoints	
Subnetwork ↑	IPv4 address	
intf-subnet	192.168.10.4	
intf-subnet	192.168.10.5	

15. Cloud Logging 驗證

Vertex AI Pipelines 工作首次執行約需 14 分鐘，後續執行時間會大幅縮短。如要驗證結果是否成功，請執行下列操作：

前往「Vertex AI」→「訓練」→「自訂工作」

選取已執行的自訂工作

Training

+ Train new model

Refresh

Learn

Training pipelinesCustom jobsHyperparameter tuning jobsNAS jobsPersistent resources

Custom jobs specify how Vertex AI runs your custom training code, including worker pools, machine types, and settings related to your Python training application and custom container. Custom jobs are only used by custom-trained models and not AutoML models. [Learn more](#)

Region

us-central1 (Iowa)

Filter

Enter a property name

Name	ID	Status	Job type	Duration	Last update	
test_psci-nonRFC_20250925210435	4261092115842334720	Finished	Custom job	30 sec	Sep 25, 2025 2:05:35 PM	

選取「查看記錄」

test_psci-nonRFC_20250925210435

Custom job was completed on Sep 25, 2025, 2:05:34 PM.

Status	Finished
Custom job ID	4261092115842334720
Created	Sep 25, 2025, 2:04:37 PM
Start time	Sep 25, 2025, 2:05:04 PM
Elapsed time	30 sec
Region	us-central1
Encryption type	Google-managed
Machine type (Worker pool 0 (chief))	n1-standard-4
Machine type (Worker pool 0 (chief))	1
Container Location (Worker pool 0 (chief))	us-central1-docker.pkg.dev/named-chariot-352806/pipelines-test-repo-psc/nonrfc-ip-call:v1
Algorithm	Custom training
Objective	Custom
Container (Training)	Custom
Logs	View logs

[View Custom Job inputs in JSON](#)

CPU

GPU

Network

Collapse all

Expand all

CPU utilization

Percent

Cloud Logging 可用後，請選取「執行查詢」，產生下方醒目顯示的選取項目，確認從 Vertex AI Pipelines 到 `class-e-vm` 的 `wget` 成功。

Project logs

Search all fields

All resources

gateway_requests

Info

Correlate by

1 logName="proj...

2 severity="INFO

Select log names

Clear

X

gate

NETWORK SERVICES API

gateway_requests

networkservices.googleapis.com/gateway_req...

Example queries

Fields

Search fields a

System Metadata

Severity

Showing top 1 of

Info

Resource type

JSON payload (most

@type

enforcedGatew

Cancel

Apply

```

{
  httpRequest: {
    latency: "0.011803s"
    protocol: "HTTP/1.1"
    remoteIp: "192.168.10.2:41200"
    requestMethod: "GET"
    requestSize: "176"
    requestUrl: "http://class-e-vm.demo.com/"
    responseSize: "245"
    serverIp: "240.0.0.2:80"
    status: 200
    userAgent: "python-requests/2.32.5"
  }
  insertId: "sbpf04e5156c"
  jsonPayload: {
    @type: "type.googleapis.com/google.cloud.loadbalancing.type.LoadBalancerLogEntry"
    enforcedGatewaySecurityPolicy: {
      hostname: "class-e-vm.demo.com"
      matchedRules: [
        {
          action: "ALLOWED"
          name: "projects/232347644611/locations/us-central1/gatewaySecurityPolicies/vertexai-pupi/rules/test"
        }
      ]
    }
  }
}

```


16. TCPDump 驗證

讓我們來檢查 TCPDUMP 輸出內容，進一步驗證與運算執行個體的連線：

從 `class-e-vm` 觀察 HTTP GET 和 200 OK

```
XXXXXXXXXX@class-e-vm:~$ sudo tcpdump -i any net 10.10.100.0/28 -nn
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144
bytes
05:51:14.173641 ens4 In IP 10.10.100.8.55306 > 240.0.0.2.80: Flags [S], seq
1747181041, win 65535, options [mss 1420,sackOK,TS val 3942828403 ecr 0,nop,wscale
8], length 0
05:51:14.173668 ens4 Out IP 240.0.0.2.80 > 10.10.100.8.55306: Flags [S.], seq
3013226100, ack 1747181042, win 64768, options [mss 1420,sackOK,TS val 1886125065 ecr
3942828403,nop,wscale 7], length 0
05:51:14.174977 ens4 In IP 10.10.100.8.55306 > 240.0.0.2.80: Flags [.], ack 1, win
1054, options [nop,nop,TS val 3942828405 ecr 1886125065], length 0
05:51:14.175066 ens4 In IP 10.10.100.8.55306 > 240.0.0.2.80: Flags [P.], seq 1:223,
ack 1, win 1054, options [nop,nop,TS val 3942828405 ecr 1886125065], length 222:
HTTP: GET / HTTP/1.1
05:51:14.175096 ens4 Out IP 240.0.0.2.80 > 10.10.100.8.55306: Flags [.], ack 223,
win 505, options [nop,nop,TS val 1886125066 ecr 3942828405], length 0
05:51:14.239042 ens4 Out IP 240.0.0.2.80 > 10.10.100.8.55306: Flags [P.], seq 1:246,
ack 223, win 505, options [nop,nop,TS val 1886125130 ecr 3942828405], length 245:
HTTP: HTTP/1.1 200 OK
```

17. 清理

注意：Vertex AI Pipeline 至少一小時未使用後，您即可刪除 PSC 網路連結和子網路。

在 Cloud Shell 中刪除教學課程元件。

```
gcloud workbench instances delete workbench-tutorial --project=$projectid --location=us-central1-a

gcloud network-security gateway-security-policies rules delete allow-nonrfc-classe \
  --gateway-security-policy=policy1 \
  --location=us-central1

gcloud network-security gateway-security-policies rules delete allow-apache2 \
  --gateway-security-policy=policy1 \
  --location=us-central1

gcloud network-security gateway-security-policies delete policy1 \
  --location=us-central1
gcloud network-services gateways delete swp1 \
  --location=us-central1

gcloud compute network-attachments delete psc-network-attachment --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet rfc1918-subnet1 --region=us-central1 --quiet

gcloud dns record-sets delete class-e-vm.demo.com --zone=private-dns-codelab --type=A
gcloud dns record-sets delete explicit-swp.demo.com --zone=private-dns-codelab --type=A

gcloud dns managed-zones delete private-dns-codelab

gcloud computeinstances delete class-e-vm --project=$projectid --zone=us-central1-a --quiet
gcloud compute networks delete consumer-vpc --quiet
```

18. 恭喜

恭喜！您已成功設定並驗證 Vertex AI Pipelines Private Service Connect 介面與非 RFC IP 範圍之間的連線，並透過 Secure Web Proxy 進行連線。

您已建立消費者基礎架構，並新增網路附件，讓生產者可以建立多重 NIC VM，做為消費者和生產者通訊的橋樑。您已瞭解如何在用戶虛擬私有雲網路中部署明確的 Proxy，建立 DNS 對等互連，允許連線至無法直接從 Vertex 路由的 `class-e-vm` 執行個體。



後續步驟

延伸閱讀和影片

- [Private Service Connect 總覽](#)

What is Private Service Connect?

Google Cloud Tech

Private Service Connect

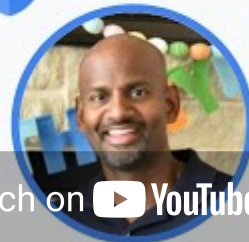


Watch on YouTube

Vertex AI networking patterns - PSC & Google APIs demo

Advanced Networking Demo

Connect to Vertex AI using PSC & Googleapis



Watch on YouTube

- [關於 Private Service Connect 介面 | VPC | Google Cloud](#)

參考文件

- [Vertex AI 網路存取權總覽 | Google Cloud](#)
- [透過 Private Service Connect 介面存取 Vertex AI 服務 | Google Cloud](#)
- [使用 Vertex AI 訓練 的 Private Service Connect 介面 | Google Cloud](#)
- [為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud](#)
- 您也可以將服務帳戶與主機位址或 IP 位址等建立關聯，以便在明確的路由模式中將流量轉送至 SWP。如需更多範例，請參閱 CEL 語言[文件](#)。